Real-Time Anomaly Detection Platform

Checkpoint 2.1 — Architecture Revision

Amen Bank — Direction Centrale du Système d'Information

June 1, 2026

Purpose of This Document

This checkpoint captures all architectural changes made to the initial system design (Checkpoint 2) following the formal internship subject issued by Amen Bank. It serves as a delta document — only deviations and additions from the original architecture are recorded here. The original Checkpoint 2 remains the canonical reference for unchanged components.

Context Shift

From Generic User Behavior to Banking Operations

The original architecture targeted generic user activity streams (logins, navigation, API calls). The revised scope is narrowed to **four specific branch teller operations** at Amen Bank:

1. **Retrait espèces** — Cash withdrawal
2. **Versement espèces** — Cash deposit
3. **Virement bancaire** — Bank transfer
4. **Remise de chèques** — Check deposit

All operations occur at the *guichet* (teller window), involving face-to-face interaction between a bank employee and a client. This is not ATM or mobile banking monitoring.

Dual-Actor Model

The original design assumed a single actor per event (the user). The revised model introduces **two actors per transaction**:

- **Client** — the account holder initiating or receiving the operation.
- **Employee** — the bank teller processing the operation.

Anomalies can originate from either actor. A client may exhibit suspicious transaction patterns; an employee may process fraudulent operations across multiple clients.

Architecture Changes

Change 1 — Kafka Replaced by RedPanda

Aspect	Checkpoint 2 (Original)	Checkpoint 2.1 (Revised)

Message broker	Apache Kafka	RedPanda
Justification	—	Kafka-compatible API, no JVM/ZooKeeper dependency, single binary. Branch operations produce lower event volumes (dozens/hour/branch vs. thousands/second), making full Kafka unnecessary.
Replay capability	Yes	Yes (log-based, same as Kafka)
Topic topology	Unchanged	Unchanged (raw-events, enriched-events, scored-events, decisions)

Change 2 — Employee Profile Service

A new profile type is introduced alongside the existing client profile. Both follow the hot/cold pattern established in Checkpoint 2.

Aspect	**Client Profile**	**Employee Profile**
Key	`client_id`	`employee_id`
Scope	Individual transaction history	Aggregated behavior across all clients served
Hot store	Redis (sub-ms lookup)	Redis (sub-ms lookup)
Cold store	PostgreSQL (full history)	PostgreSQL (full history)

Employee Profile — Feature Categories

- **Identity & context:** `employee_id`, `branch_id`
- **Volume metrics:** Operation count and monetary volume over rolling windows (daily, weekly, monthly), compared against branch peers via **z-score** (standard deviations from peer mean).
- **Near-threshold ratio:** Proportion of transactions in the 8,000–9,999 DT band (structuring/smurfing indicator, regulatory threshold ≈ 10,000 DT).
- **Client-location mismatch rate:** Frequency of processing transactions for clients whose registered branch differs from the employee's branch.
- **Error/reversal rate:** Correction and reversal frequency compared to branch peer average (z-score).
- **Client concentration:** Unusual repeat interactions with the same client beyond expected frequency.

Change 3 — SHAP Integrated into Scoring Service

Aspect	**Detail**
Placement	Inside the Scoring Service, co-located with the model.

Rationale	SHAP requires access to the trained model weights and the input feature vector at prediction time. A separate downstream consumer would lack both.
Execution policy	SHAP runs only on events exceeding the anomaly score threshold (not on every transaction) to manage latency.
Output	Feature attributions appended to the `scored-events` payload. Carried through the Decision & Alert Service into the `decisions` topic.
Dashboard impact	The Compliance Dashboard consumes a single input (the `decisions` topic). The payload now includes SHAP explanations alongside the decision, score, and original event. No second input stream required.

Change 4 — Raw Event Schema

The `raw-events` topic payload is revised to reflect banking operations:

Field	Description
`event_id`	Unique transaction identifier
`timestamp`	Operation timestamp
`operation_type`	One of: `retrait`, `versement`, `virement`, `remise_cheque`
`client_id`	Account holder identifier
`employee_id`	Teller processing the operation
`branch_id`	Branch where the operation occurs
`amount`	Transaction amount (DT)
`client_usual_branch`	Client's registered/primary branch
`beneficiary_id`	(virement only) Recipient account
`cheque_emitter_id`	(remise_cheque only) Check issuer

Change 5 — Alert Severity Levels

The Decision & Alert Service now classifies alerts into severity tiers rather than binary anomaly/normal:

- **INFO** — Minor deviation, logged but no action required.
- **REVIEW** — Requires supervisor attention (e.g., client operating from unusual branch).
- **ALERT** — Significant anomaly, immediate review recommended.
- **BLOCK** — Operation should be halted pending investigation.

This tiered approach mitigates alert fatigue, a key concern raised in the supervisor's requirements.

Change 6 — Supervisor Feedback Loop

A new data flow is introduced from the Compliance Dashboard back into the system:

1. Supervisor reviews an alert on the dashboard.
2. Supervisor marks the alert as **confirmed** (true positive) or **rejected** (false positive).
3. Feedback is written to PostgreSQL as labeled data.

4. The Batch Pipeline consumes labeled feedback during model retraining.
5. Confirmed-normal events contribute to profile rebuilds; confirmed-anomalous events are excluded from profile updates (maintaining the verified-only update policy from Checkpoint 2).

Change 7 — Synthetic Data Strategy

Aspect	Training Data	Live Demo / Evaluation
Tool	Faker / SDV	Custom event simulator
Format	Static CSV/Parquet dataset	Streaming events via RedPanda
Purpose	Model training & evaluation	End-to-end pipeline demonstration
Anomaly injection	Controlled, labeled	Real-time, configurable scenarios

Unchanged Components

The following elements from Checkpoint 2 remain unchanged:

- Four-topic RedPanda topology (raw-events → enriched-events → scored-events → decisions)
- Hot/cold profile pattern (Redis + PostgreSQL)
- Service isolation principles (fault isolation, independent scaling, security boundaries)
- Batch pipeline structure (profile rebuild, model retrain, model evaluation, drift detection)
- Profile update safety (verified-only updates, decay windows, rate-limited drift)
- Enrichment Service role and position in pipeline
- Decision & Alert Service role and position in pipeline

Updated Technology Stack

Category	Technologies
Language	Python
API Framework	FastAPI
Message Broker	RedPanda (Kafka-compatible)
Deep Learning	PyTorch (Autoencoder, LSTM)
Classical ML	Scikit-learn (Isolation Forest, LOF — baselines)
Explainability	SHAP
Synthetic Data	Faker, SDV
Hot Store	Redis
Cold Store / DWH	PostgreSQL
Dashboard	Streamlit, Plotly

Experiment Tracking	MLflow
Containerization	Docker
Monitoring	Prometheus, Grafana

Next Steps

1. Finalize the *document de cadrage* and *glossaire métier* (Week 1 deliverable).
2. Design the complete data schemas: raw event, enriched feature vector, client profile, employee profile.
3. Build the synthetic data generator covering all four operation types with realistic anomaly injection.
4. Begin incremental service implementation starting with the Ingestion Service.